# Randomized Algorithm

Pritha Banerjee

University of Calcutta

*banerjee.pritha74@gmail.com*

October 17, 2025

# Overview

1. Randomized Algorithm
2. Randomized QuickSort
3. Randomized Selection
4. Closest Pair
   - Using Divide and Conquer
   - Using Randomization
   - Design of Algorithm
   - Data structure for Subsquares
     - Hashing: A Randomization Implementation of Dictionary
   - Time Complexity Analysis
5. Books etc.

# Deterministic Algorithm

- On input $A$, determinstic algorithm must produce a correct solution
- The number of computation steps must be same for different runs of algo with same input
- It may be difficult to formulate an algorithm with good runtime for sufficiently large input

**How to get solutions quickly? Randomized Algorithm**

# Randomized Algorithm (1)

- On input $A$, randomized algorithm uses random choices to produce the correct solution
- It may go through different steps for different runs of algorithm with same input due to random choices.
- **Advantages:**
  - Randomized algorithms are often simple and easy to implement
  - Usually fast with high probability compared to deterministic algorithms.
  - Produces optimum output with very high probability
  - Malicious adversaries cannot generate any worst case input, that would slow down the randomized algorithm
- **Disadvantages:**
  - There is a finite probability of getting incorrect answer. But the probability can be made arbitrarily small by repeated application of randomness
  - Analysis of runtime and probability of getting correct answer is complex
  - Getting true random number is impossible and results depends on quality of random numbers.

# Randomized Algorithm (2)

- Compute expected running time of randomized algorithm, as runtime is independent of any particular input.
- Inputs follow a probability distribution; thus, probabilistic analysis of expected running time

**Find an $a$ in input $A$ where $n/2$ elements are $a$ and rest $b$ : Las Vegas Vs. Monte Carlo Algorithm**

```
 findingA_LV(array A, n)
begin
    repeat
        Randomly select one element out of n elements.
    until 'a' is found
end

findingA_MC(array A, n, k)
begin
    i := 0
    repeat
        Randomly select one element out of n elements.
        i := i + 1
    until i = k or 'a' is found
end
```

# Randomized Algorithm (3)

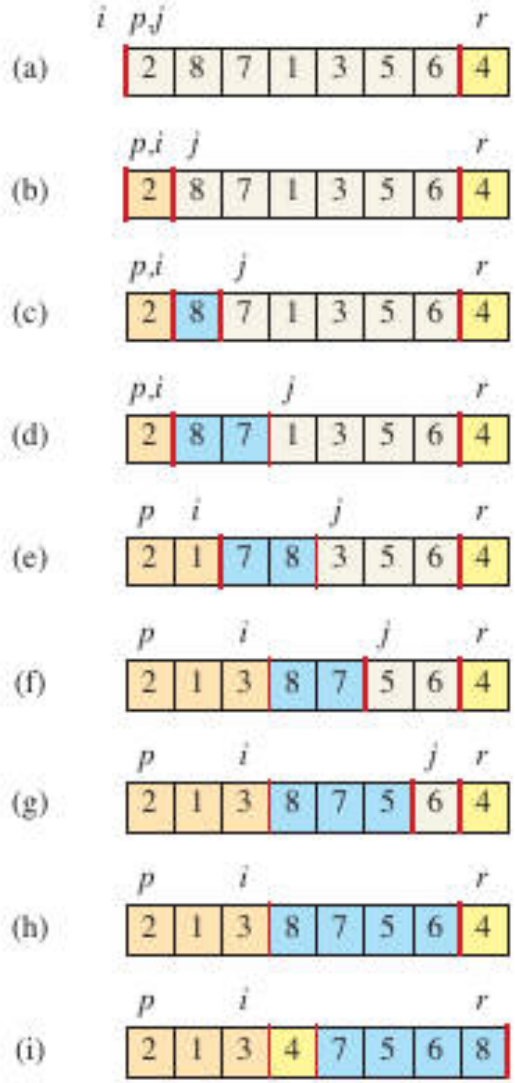
**findingA(LV):**

- Algo succeeds with probability 1, though number of iterations may vary.
- Expected number of iterations: $\lim_{n\to\infty} \sum_{i=1}^{n} \frac{i}{2^i} = 2$, ie, $\Theta(1)$

**findingA(MC):**

- if an $a$ is found algo. succeeds, else it fails.
- After$k$ iterations, the probability of finding an 'a': $P\left[find\ a\right] = 1 - (\frac{1}{2})^k$
- The number of iterations is always less than or equal to $k$. $k$ being constant the run time (expected and absolute) is $\Theta(1)$.

# Quicksort




```
QUICKSORT(A, p, r)
1  if p < r
2      // Partition the subarray around the pivot, which ends up in A[q].
3      q = PARTITION(A, p, r)
4      QUICKSORT(A, p, q − 1)  // recursively sort the low side
5      QUICKSORT(A, q + 1, r)  // recursively sort the high side
```

```
PARTITION(A, p, r)
1  x = A[r]                          // the pivot
2  i = p − 1                         // highest index into the low side
3  for j = p to r − 1                // process each element other than the pivot
4      if A[j] ≤ x                   // does this element belong on the low side?
5          i = i + 1                 // index of a new slot in the low side
6          exchange A[i] with A[j]   // put this element there
7  exchange A[i + 1] with A[r]       // pivot goes just to the right of the low side
8  return i + 1                      // new index of the pivot
```

# Randomized Quicksort

- Instead of always using $A[r]$ as pivot, select a randomly chosen element from $A[p..r]$

```
RANDOMIZED-QUICKSORT(A, p, r)
1  if p < r
2      q = RANDOMIZED-PARTITION(A, p, r)
3      RANDOMIZED-QUICKSORT(A, p, q − 1)
4      RANDOMIZED-QUICKSORT(A, q + 1, r)
```

```
RANDOMIZED-PARTITION(A, p, r)
1  i = RANDOM(p, r)
2  exchange A[r] with A[i]
3  return PARTITION(A, p, r)
```

```
PARTITION(A, p, r)
1  x = A[r]
2  i = p − 1
3  for j = p to r − 1
4      if A[j] ≤ x
5          i = i + 1
6          exchange A[i] with A[j]
7  exchange A[i + 1] with A[r]
8  return i + 1
```

# Randomized Quicksort: Worst case time complexity (1)

- $T(n) = \max_{0 \le q \le n-1} (T(q) + T(n-q-1)) + \Theta(n)$
- solve by substitution method: Assume $T(n) \le cn^2$
- substitute the guess, ie,

$$\begin{aligned} T(n) &\le \max_{0 \le q \le n-1} (cq^2 + c(n-q-1)^2) + \Theta(n) \\ &= c \max_{0 \le q \le n-1} (q^2 + (n-q-1)^2) + \Theta(n) \end{aligned}$$

- $q^2 + (n-q-1)^2$ is maximum when $q = 0$ or $q = n-1$. For $q = 0, 1, \cdots, n-1$

$$\begin{aligned} q^2 + (n-q-1)^2) &= q^2 + (n-1)^2 - 2q(n-1) + q^2 \\ &= (n-1)^2 + 2q(q-(n-1)) \\ &\le (n-1)^2, \because q \le (n-1), 2q(q-(n-1)) \le 0 \end{aligned}$$

- Every term in the maximization is bounded by $(n-1)^2$

# Randomized Quicksort: Worst case time complexity (2)

- $\max_{0 \le q \le n-1} (q^2 + (n-q-1)^2) \le (n-1)^2 = n^2 - 2n + 1$
- $T(n) \le cn^2 - c(2n-1) + \Theta(n)$
- since $c$ can be taken large to dominate $\Theta(n)$, so $T(n) = O(n^2)$
- when partition is maximally unbalanced, $T(n) = \Omega(n^2)$, Thus, Worst case time complexity is $\Theta(n^2)$
- Randomized Quicksort is as bad as Quicksort for in the worst case.

# Randomized Quicksort:Expected case time complexity (1)

- *PARTITION* is called $n$ times throughout the quicksort, as during every partition a pivot is chosen and that pivot never appears later.
- Let $X$ is the total number of comparisons in the entire quicksort algorithm (line 4 in Partition)
- Then the running time of QUICKSORT is $O(n + X)$
- Let $A$ has elements $z_1, z_2, \cdots, z_n$, where $z_i$ is the $i^{th}$ smallest element, so, define $Z_{ij} = \{z_i, z_{i+1}, \cdots, z_j\}$
- Need to know **when the algo. compares two elements $z_i$ and $z_j$ in the array and when it does not**
  - note that each pair of elements are compared only once
  - pivot is compared with all the elements only once, as that pivot does not participate later, once it is placed in correct position
- need probabilistic analysis: Let $X_{ij}$ is indicator random variable

$$X_{ij} = 1, \text{when } z_i \text{ and } z_j \text{ is compared} \quad (1)$$

$$= 0, \text{otherwise} \quad (2)$$

# Randomized Quicksort:Expected case time complexity (2)

- since each pair compared only once, total number of comparisons $X$ is

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

- taking expectation,

$$E(X) = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}\right]$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} E\left[X_{ij}\right]$$

$$= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} Pr\{z_i \text{ is compared to } z_j\}$$

# Randomized Quicksort:Expected case time complexity (3)

- Note 1): once a pivot $x$ is chosen with $z_i < x < z_j$, then $z_i$ and $z_j$ are never compared any later time.
- Note 2): if $z_i$ or $z_j$ is pivot, then $z_i$ or $z_j$ is compared with any other item in $Z_{ij}$, except themselves
- what is the probability that $z_i$ or $z_j$ from $Z_{ij}$ is the first element chosen as a pivot?
- any element of $Z_{ij}$ is equally likely to be the first one chosen as pivot
- Pr $\{$ any elemnet is first one chosen as pivot from $Z_{ij}$ $\} = \frac{1}{(j-i+1)}$

$$
\begin{aligned}
Pr\{ z_i \text{ is compared to } z_j \} &= Pr\{ z_i \text{ or } z_j \text{ is first pivot chosen from } Z_{ij}\} \\
&= Pr\{ z_i \text{ is first pivot chosen from } Z_{ij}\} \\
&+ Pr\{ z_j \text{ is first pivot chosen from } Z_{ij}\} \\
&= \frac{1}{(j-i+1)} + \frac{1}{(j-i+1)}, \quad \text{mutually exclusive} \\
&= \frac{2}{(j-i+1)}
\end{aligned}
$$

# Randomized Quicksort:Expected case time complexity (4)

$$
\begin{aligned}
E(X) &= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{(j-i+1)}, \text{ harmonic series} \\
&= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{(k+1)}, \; k = j-i \\
&< \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{(k)} \\
&= \sum_{i=1}^{n-1} O(\log n) = O(n \log n)
\end{aligned}
$$

# Randomized Selection

**Selection Problem: find $k^{th}$ smallest element from $A[1..n]$**

```
RANDOMIZED-SELECT(A, p, r, i)
1   if p == r
2       return A[p]
3   q = RANDOMIZED-PARTITION(A, p, r)
4   k = q − p + 1
5   if i == k            // the pivot value is the answer
6       return A[q]
7   elseif i < k
8       return RANDOMIZED-SELECT(A, p, q − 1, i)
9   else return RANDOMIZED-SELECT(A, q + 1, r, i − k)
```

# Randomized Selection : Time complexity (1)

- Worst case: $\Theta(n^2)$, if partitions always with the largest remaining element
- Expected running time: $O(n)$
- Run time $T(n)$ on input array $A[p..r]$ is a **random variable**; compute $E[T(n)]$
- Randomized partition is equally likely to return any element as pivot
- Thus, for each $k$ such that $1 \leq k \leq n$, the subarray $A[p..q]$ has $k$ elements $\leq$ pivot with probability $\frac{1}{n}$
- Let $X_k = 1$; $\{$the subarray $A[p..q]$ has exactly $k$ elements $\}$, else $X_k = 0$
- $E[X_k] = P[X_k = 0].0 + P[X_k = 1].1 = \frac{1}{n}.1 = \frac{1}{n}$
- Compute an upper bound on time needed for recursive call on largest possible partition size $A[p..q-1]$ or $A[q+1..r]$

# Randomized Selection : Time complexity (2)

- $X_k$ has value 1 for exactly one value of $k$.

$$
\begin{aligned}
T(n) &\leq \sum_{k=1}^{n} X_k.(T(max(k-1, n-k)) + O(n)) \\
&= \sum_{k=1}^{n} X_k.T(max(k-1, n-k)) + \sum_{k=1}^{n} X_k.O(n) \\
&= \sum_{k=1}^{n} X_k.T(max(k-1, n-k)) + O(n); \ \because X_k = 1 \text{ for one } k
\end{aligned}
$$

# Randomized Selection : Time complexity (3)

$$E[T(n)]$$

$$\leq E\left[\sum_{k=1}^{n} X_k.T(max(k-1,n-k)) + O(n)\right]$$

$$= \sum_{k=1}^{n} E[X_k.T(max(k-1,n-k))] + O(n); \text{ linearity of Expectation}$$

$$= \sum_{k=1}^{n} E[X_k].E[T(max(k-1,n-k))] + O(n); X_k \text{ and } T(max()) \text{ independent rv}$$

$$= \sum_{k=1}^{n} \frac{1}{n}.E[T(max(k-1,n-k))] + O(n); \text{ using } E[X_k] = \frac{1}{n}$$

**What is $max(k-1,n-k)$?**

$$max(k-1,n-k) = \begin{cases} k-1 & \text{if} k > \lceil n/2 \rceil \\ n-k & \text{if} k \leq \lceil n/2 \rceil \end{cases} \quad (3)$$

# Randomized Selection : Time complexity (3)

Note that each term appears twice in $T(max(k-1, n-k))$, from $T(\lceil n/2 \rceil)$ to $T(n-1)$ for $k = 1 \cdots n$, thus,

$$= \frac{1}{n}(E[T(n-1)] + E[T(n-2)] \cdots + E[T(n/2+1)] +$$

$$E[T(n/2)+1] \cdots E[T(n-1)]) + O(n)$$

$$\text{Thus, } E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n)$$

Use substitution to show $E[T(n)] = O(n)$, assume $E[T(n)] \leq cn$ and $T(n) = 1$ for $n$ less that some constant $E[T(n)]$

$$\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an$$

$$= \frac{2c}{n} \left( \sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) + an$$

# Randomized Selection : Time complexity (4)

$$
\begin{aligned}
&= \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1)\lfloor n/2 \rfloor}{2}\right) + an \\
&\leq \frac{2c}{n}\left(\frac{(n-1)n}{2} - \frac{(n/2-2)(n/2-1)}{2}\right) + an \\
&= \frac{2c}{n}\left(\frac{n^2-n}{2} - \frac{n^2/4 - 3n/2 + 2}{2}\right) + an \\
&= \frac{c}{n}\left(\frac{3n^2}{4} + \frac{n}{2} - 2\right) + an \\
&= c\left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n}\right) + an \\
&\leq \frac{3cn}{4} + \frac{c}{2} + an \\
&= cn - (\frac{cn}{4} - \frac{c}{2} - an)
\end{aligned}
$$

- To show for large $n$ last expression is atmost $cn$, ie, $\frac{cn}{4} - \frac{c}{2} - an \geq 0$
- Add $c/2$ to both sides and factor out $n$ to get $n(c/4 - a) \geq c/2$, ie, $(c/4 - a) > 0$, ie, $c > 4a$
- Thus, $n \geq \frac{c/2}{c/4-a} = \frac{2c}{c-4a}$
- For such $n$ and $c$, $E[T(n)] = O(n)$

# D & C Closest Pair Problem

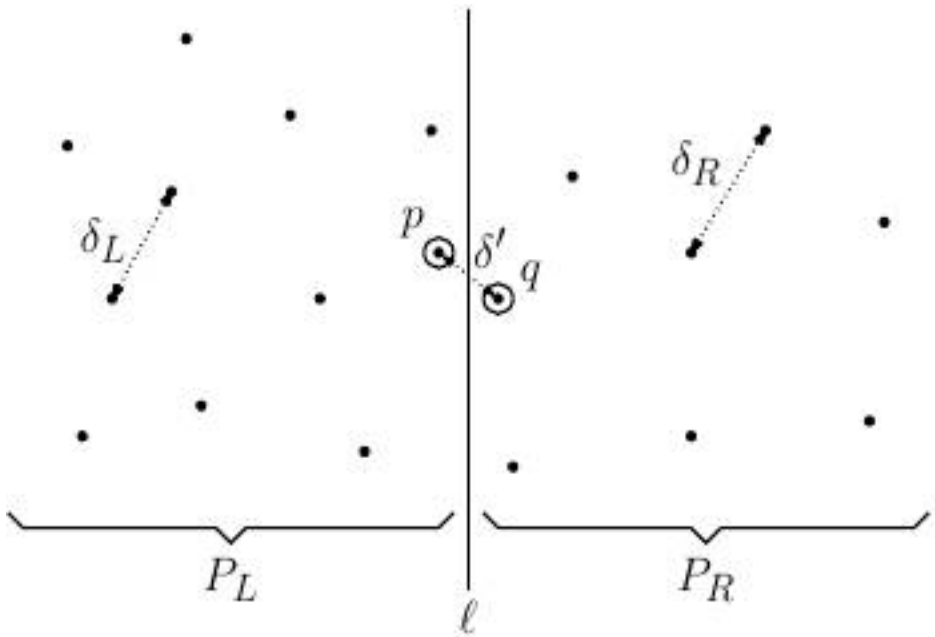
**Problem:** Given a set of points $P = \{p_1, p_2, \cdots, p_n\}$, where each $p_i$ is $(x_i, y_i)$, find the closest pair. Here closest pair is a pair of points whose Euclidean distance is minumum among all possible pairs.

- Given two points $p = (p_x, p_y)$ and $q = (q_x, q_y)$, their (Euclidean) distance is $||pq|| = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$.
- Brute force search: Takes $O(n^2)$ time, by computing the distance between each pair, and returning the smallest.
- Goal: $O(n \log n)$ time algorithm based on divide-and-conquer.

# D & C Closest Pair Algorithm

- **Basis:** If $|P| \leq 3$, then just solve the problem by brute force in $O(1)$ time
- **Divide:** Otherwise, partition the points into two partitions $P_L$ and $P_R$ of roughly equal size (find median) based on their $x$-coordinates. Imagine a vertical line $l$ that splits the points to $P_L$ with points to the left of $l$ and $P_R$, the points to the right of $l$.
- **Conquer:** Compute the closest pair within each of the subsets $P_L$ and $P_R$ each, by invoking the algorithm recursively. Let $\delta_L$ and $\delta_R$ be the closest pair distances in each case Let $\delta = min(\delta_L, \delta_R)$.
- **Combine:** Note that there may be two points $p$ and $q$ that are very close to one another but are on opposite sides of $l$. Need to find such closest pair $(p, q)$ if exists with distance $\delta' < \delta$. If found, return $min(\delta, \delta')$ as the final result along with $p$, $q$ being the closest.

# D & C: Idea

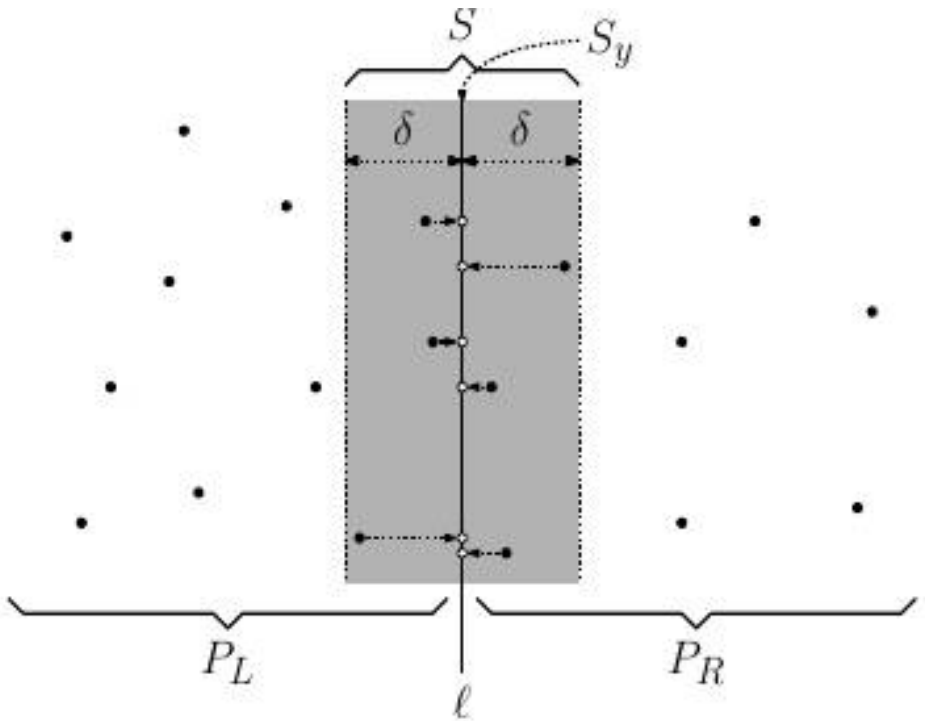



- **How to find median during each recursion (divide) ?** Expensive even if *Select* algorithm is used during every recursion. Hence **pre-sort** the $x$ coordinate ($P_x$) and $y$ coordinate ($P_y$) in $O(\log n)$ time only **once**. Use required subarray of $P_x$ to find median during each recursion. In $O(n)$, partition the points into $P_L$ and $P_R$
- If **combine** step can be done in $O(n)$ time, then time complexity is $T(n) = 2T(n/2) + O(n) = O(n \log n)$

# How to combine? (1)

Need to compute the closest pair $p$ and $q$, where $p \in P_L$ and $q \in P_R$.

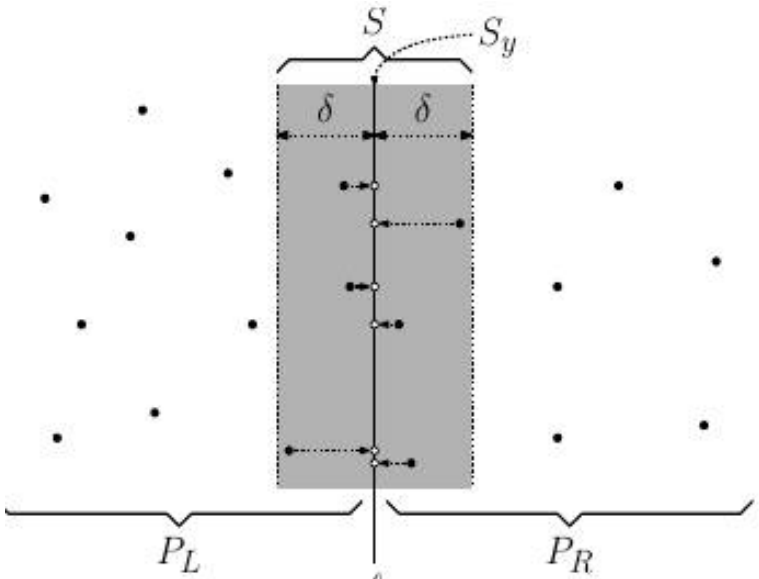
- We have the point set $P$ , the x-coordinate of the vertical splitting line $l$, and the value of $\delta = min(\delta_L, \delta_R)$.
- Note that if such a pair of points $p$, $q$ exists, then both points lie within distance $\delta$ of $l$, for otherwise the resulting distance would exceed $\delta$.
- Let $S$ denote this subset of $P$ that lies within a vertical strip of width $2\delta$ centered about $l$

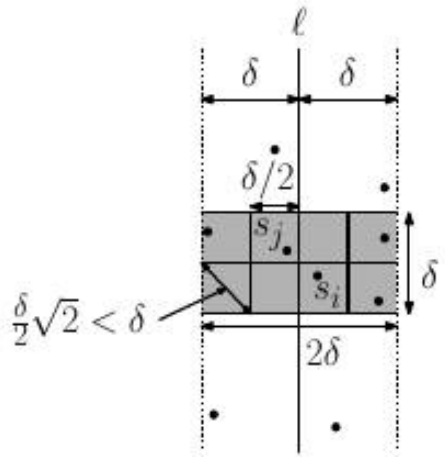


# How to combine? (2)

How to find the closest pair within $S$?

- Let $Sy =< s_1, \cdots, s_m >$ are points of $S$ sorted by their $y$-coordinates.
- If $S_y$ contains two points that are within distance $\delta$ of each other, these two points must be within a **constant number of positions of each other** in the sorted array $S_y$.
- **Lemma:** Given any two points $s_i, s_j \in S_y$ , if $||s_i s_j|| \leq \delta$, then $|j - i| \leq 7$.
- On the **sorted** $S_y$, compute distance $\delta'$ of a point with its 8 adjacent points in increasing order of $S_y$. If $\delta' < \delta$ update closest points and closest distance. Thus $O(n)$ time, if $n$ points are in $S_y$



# How to combine? (3)

**Lemma:** Given any two points $s_i, s_j \in S_y$ , if $||s_i s_j|| \leq \delta$, then $|j - i| \leq 7$.
**Proof:**

- Suppose that $||s_i s_j|| \leq \delta$. Since they are in $S$ they are each within distance $\delta$ of $l$.
- The $y$-coordinates of these two points can differ by at most $\delta$. So they must both reside in a rectangle of width $2\delta$ and height $\delta$ centered about $l$.
- Split this rectangle into **eight** identical squares each of side length $\delta/2$. A square of side length $\delta/2$ has a diagonal of length $\sqrt{(\delta/2)^2 + (\delta/2)^2} = \delta/\sqrt{2} < \delta$, and so no two points within such a square can be farther away than this.
- Since each square lies entirely on one side of $l$, no square can contain two or more points of $P$ , since otherwise, these two points would contradict the fact that $\delta$ is the closest pair seen so far.
- Thus, there can be at most **eight** points of $S$ in this rectangle, one for each square. Therefore, $|j - i| \leq 7$.

# Computing $S_y$

- Sorting $S_y$ explicitly is expensive, because we may have $n$ points in $S$ and we want $O(n)$ time for combine step
- Pre-sorting $P_y$ helps.
- Compute $S_y$, enumerate the points of $P_y$, and each time we find a point that lies within the **strip**, copy it to the next position of array $S_y$ . This runs in $O(n)$ time, and preserves the $y$-ordering of the points.

# D & C Closest Pair Algo

```
closestPair(P = (Px, Py)) {
    n = |P|
    if (n <= 3) solve by brute force                    // basis case
    else {
        Find the vertical line L through P's median     // divide
        Split P into PL and PR (split Px and Py as well)
        dL = closestPair(PL)                            // conquer
        dR = closestPair(PR)
        d = min(dL, dR)
        for (i = 1 to n) {                              // create Sy
            if (Py[i] is within distance d of L) {
                append Py[i] to Sy
            }
        }
        d' = stripClosest(Sy)                           // closest in strip
        return min(d, d')                               // overall closest
    }
}

stripClosest(Sy) {                                      // closest in strip
    m = |Sy|
    d' = infinity
    for (i = 1 to m) {
        for (j = i+1 to min(m, i+7)) {                  // search neighbors
            if (dist(Sy[i], Sy[j]) <= d') {
                d' = dist(Sy[i], Sy[j])                 // new closest found
            }
        }
    }
    return d'
}
```

# Closest Pair Problem and Randomization

**Problem:** Given a set of points $P = \{p_1, p_2, \cdots, p_n\}$, where each $p_i$ is $(x_i, y_i)$, find the closest pair.

- It uses randomization and dictionary data structure
- Algorithm runs in $O(n)$ expected time, plus $O(n)$ expected dictionary operations.
- Randomness happens as follows:
  - the way in which the algorithm processes the input points
  - when the dictionary is implemented using hashing
- Let $d(p_i, p_j)$ denotes the Euclidean distance between points $p_i$ and $p_j$. Thus, find the pairs $p_i$ and $p_j$ such that $d(p_i, p_j)$ is minimized.
- Assume that the points are all in the unit square: $0 \leq x_i, y_i < 1$, $i = 1, 2, \cdots, n$

# Designing the algorithm (1)

**Overview:**

- Consider the points in **random order**, and maintain a current value $\delta$ for closest pair
- When a new point $p$ appears, look **in the vicinity** of $p$ to see if any of the previously considered points are at distance less than $\delta$ from $p$
- **If not**, then closest pair has **not changed**, move to the next point in **random** order
- **Else**, there is a ponit closer to $p$, update $\delta$ and the closest pair

**Stages of Algorithm:**

- Assume **random order** of points are $p_1, p_2, \cdots, p_n$
- In each **stage**, closest pair remains constant
- **First stage** : $\delta = d(p_1, p_2)$
- During a **stage** points get added as long as there is **no change in closest pair**
- Stage terminates for a point $p_i$, when some $p_j, j < i$, $d(p_i, p_j) < \delta$
- Thus $\delta = \min_{j:j<i} d(p_i, p_j)$

# Designing the algorithm (2)

**Note:**

- Number of stages depends on the random order of choosing points
- Minimum 1 stage, if first two points chosen are closest
- Maximum $n-2$ stages, if for every point chosen next, there is a change in closest pair

**Testing a proposed distance:**

- Subdivide the unit square into subsquares, with side $\delta/2$, ie, $N = 2/\delta$ subsquares on each side, $N^2$ subsquares
- Subsquare $S_{st}$, for $0 \leq s \leq N-1$ and $0 \leq t \leq N-1$ is

$$S_{st} = \{(x,y) : s(\delta/2) \leq x < (s+1)(\delta/2); t(\delta/2) \leq y < (t+1)(\delta/2)\}$$

- Observe **first**, that any two points that lie in the same subsquare have distance less than $\delta$.
- **Second**, any two points that are less than $\delta$ away from each other must fall in either the same subsquare or in very close subsquares

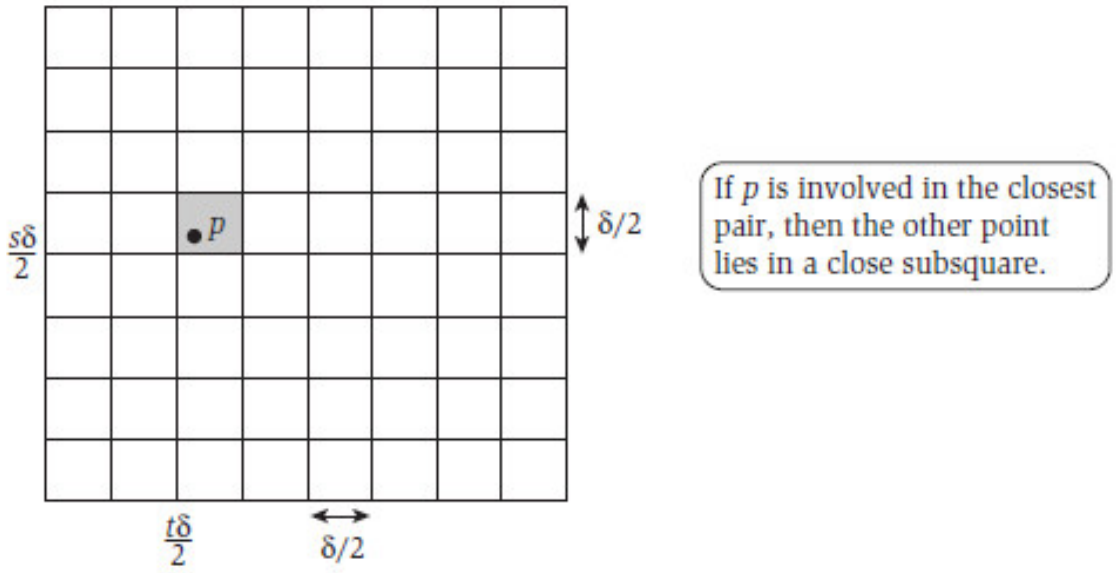
# Designing the algorithm (3): Lemma 1

**Lemma 1:** If two points $p$ and $q$ belong to the same subsquare $S_{st}$, then $d(p, q) < \delta$.

**Proof:**

- If points $p$ and $q$ are in same subsquare, then both coordinates of the two points differ by at most $\delta/2$, and hence

$$d(p, q) \leq \sqrt{(\delta/2)^2 + (\delta/2)^2} = \delta/\sqrt{2} < \delta$$



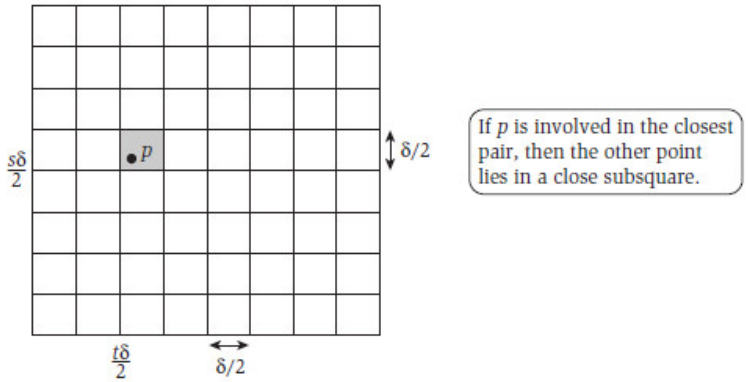

- Subsquares $S_{st}$ and $S_{s't'}$ are close if $|s - s'| \leq 2$ and $|t - t'| \leq 2$.

# Designing the algorithm (4): Lemma 2

**Lemma 2:** If for two points $p, q \in P$ we have $d(p, q) < \delta$, then the subsquares containing them are close.

**Proof:**

- Consider $p, q \in P$ belongs to subsquares that are not close
- Assume $p \in S_{st}$ and $q \in S_{s't'}$, where one of $s, s'$ or $t, t'$ differs by more than 2. ie, in one of their respective $x$ or $y$-coordinates, $p$ and $q$ differ by at least $\delta$, thus we cannot have $d(p, q) < \delta$.




- Note: for any $S_{st}$, the set of close subsquares is a $5 \times 5$ grid around it
- At most 25 subsquares need to be checked for new closest pair for a new point $p$ found in $S_{st}$

# Designing the algorithm (5): Summary

- Let $P' \subseteq P$ is set of random points already seen and $\delta$ is minimum distance seen so far for $P'$.
- For each of the points in $P'$, keep track of the subsquare containing it.
- When next point $p_i$ comes, determine which of the subsquare $S_{st}$ it belongs to
- If $p_i$ causes $\delta$ to change, there must be some earlier point $p_j \in P'$ at distance less than $\delta$ from it
- Thus, $p_j$ must be in one of the 25 squares around $S_{st}$ containing $p_i$.
- Check each of these 25 squares to see if it contains a point in $P'$ (Lemma 2)
- For each point $p_j \in P'$ in 25 subsquares, compute its distance to current point $p_i$
- Each of these subsquares contains at most one point of $P'$ (Lemma 1), ie, at most a **constant (25)** number of **distance computations**

# Designing the algorithm (6): Summary

- If a new $\delta'$ (smaller distance) is found, discard the old subsquares, else **phase** continues
- **If new** $\delta'$, update points of $p_j \in P'$ and $p_i$ in newly formed subsquares of new $\delta'/2$ side.
- Continue unless all points are processed

# Randomized Closest Pair Algorithm

Order the points in a random sequence $p_1, p_2, \ldots, p_n$
Let $\delta$ denote the minimum distance found so far
Initialize $\delta = d(p_1, p_2)$
Invoke MakeDictionary for storing subsquares of side length $\delta/2$
For $i = 1, 2, \ldots, n$:
  Determine the subsquare $S_{st}$ containing $p_i$
  Look up the 25 subsquares close to $p_i$
  Compute the distance from $p_i$ to any points found in these subsquares
  If there is a point $p_j$ $(j < i)$ such that $\delta' = d(p_j, p_i) < \delta$ then
    Delete the current dictionary
    Invoke MakeDictionary for storing subsquares of side length $\delta'/2$
    For each of the points $p_1, p_2, \ldots, p_i$:
      Determine the subsquare of side length $\delta'/2$ that contains it
      Insert this subsquare into the new dictionary
    Endfor
  Else
    Insert $p_i$ into the current dictionary
  Endif
Endfor

# Dictionary: for storing subsquares

- Need to name a subsquare $S_{st}$ and quickly determine which points of $P$, if any, are contained in it; thus **Dictionary**
- The universe $U$ of possible elements is set of all subsquares; set $S$ maintained by the data structure are subsquares that contain points from set $P'$ that has been already seen
- For each $p_j \in P'$ keep the subsquare containing it, tagged with index of $p_j$. Note, $N^2 = (2/\delta)^2$ much larger than $n$, thus **Hashing**
- **New point** $p_i$ appears; 1) determine $S_{st}$ containing it; 2) perform a **Lookup** for each of the 25 subsquares close to $S_{st}$ for any points $pj$ using **Dictionary** and compute $d(p_i, p')$. if
  - all $d(p_i, p_j) > \delta$: insert $p_i$ in dictionary, ie. map $(s, t)$ of $p_i$ to a hash table slot tagged with $p_i$
  - any $d(p_i, p_j) < \delta, \delta = d(p_i, p_j)$: discard current **Dictionary**; create new Dictionary **MakeDictionary** with **new** $\delta/2$, ie, insert all points $p' \in P'$ and $p_i$ in corresponding subsquares in dictionary.

# Hashing: Randomized implementation of Dictionary

- Hashing is used to maintain a dynamically changing set of elements; Ex: set of news service recording first paragraph of articles
- $U$: Universe of possible elements / keys, which of very large size; $S \subseteq U$ that needs to be maintained dynamically for insert, delete and search
- Makedictionary: initializes a freash dictionary that stores $S \subseteq U$, initially empty; it supports operations:
  - **Insert(u)**: adds $u \in U$ to $S$
  - **Delete(u)**: remove $u$ from $S$
  - **Lookup(u)**: if $u \in S$ returns relevant information of $u$
- **Example:** Hash function $h(x) = x \mod 250$ maps IP addresses to positions in an array (table) of length 250.

# Families of Hash Function (1)

- Hash function must be random but consistent; however, no single hash function behaves well on all sets of data
- Example: if $2^{32}$ IP addresses to be mapped to 250 slots, atleast $2^{32}/250 \approx 2^{24}$ hashing to same slot (collision), thus **randomization**
- Pick up a hash function at random from some class of functions; this will reduce collisions.
- **Example:** Let number of slots is $n = 257$ ( a prime number close to 250). IP address $x = (x_1, x_2, x_3, x_4)$; then $h(x_1, x_2, x_3, x_4) = (87x_1 + 23x_2 + 125x_3 + 4x_4) \mod 257$, where $\{87, 23, 125, 4\} \in \{0, 1, \cdots, n\}$ be any set of 4 coefficients.
- Thus, for any $x = (x_1, x_2, \cdots, x_m)$ and $a = (a_1, a_2, \cdots, a_m) \in \{0, 1, \cdots, n\}$, define $h_a(x) = \sum_{i=1}^{m} a_i x_i \mod n$

# Families of Hash function (2)

- Any $a_i$ picked at random $h_a(x)$ is likely to be a good hash function.
- **Property:** if $x = (x_1, x_2, x_3, x_4)$ and $y = (y_1, y_2, y_3, y_4)$ are distinct, and $a = (a_1, a_2, a_3, a_4)$ are chosen uniformy at random from $\{0, 1, \cdots, n-1\}$, then $Pr\left[h_a(x_1, x_2, x_3, x_4) = h_a(y_1, y_2, y_3, y_4)\right] = \frac{1}{n}$
- **Proof:**
  - Let $x$ and $y$ differ by some component $x_4 \neq y_4$
  - Compute $Pr\left[\sum_{i=1}^{4} a_i x_i \mod n = \sum_{i=1}^{4} a_i y_i \mod n\right]$
  - Compute $Pr\left[\sum_{i=1}^{3} a_i(x_i - y_i) \mod n = a_4(y_4 - x_4) \mod n\right]$,
  - After picking up $a_1, a_2, a_3$ from $\{0, 1, \cdots n-1\}$, $a_4$, compute probability such that $\sum_{i=1}^{3} a_i(x_i - y_i) \mod n = a_4(y_4 - x_4) \mod n$ holds
  - Let $c = \sum_{i=1}^{3} a_i(x_i - y_i)$, $n$ is prime and $x_4 \neq y_4$
  - Then $a_4 = c(y_4 - x_4)^{-1} \mod n$, probability of this is $\frac{1}{n}$
- Familiy $\mathcal{H}$ of Universal Hash function $\mathcal{H} = \{h_a : a \in \{0, 1, \cdots, n-1\}^m\}$

# Application of Universal Hash function

- Very few subsquares $S_{st}$ from the universe $U$ of $N^2$ subsqares will have points in it.
- Thus, universal hash function $h_a(s,t) = (a_1 s + a_2 t) \mod n$ is used to map subsquare $S_{st}$ to a hash table slot, where $p_i$ is tagged.
- **MakeDictionary()** maps each $p' \in P'$ to the corresponding hash table slot using $(s,t)$, the subsquare $S_{st}$ that $p'$ belongs to

# Time complexity (1)

- **Observation 1:** We need atmost $n$ **MakeDictionary**, the worst case, when new $\delta$ is found for every point that appears, ie, $O(n)$
- **Observation 2:** a) Algo. maintains closest pair at all times, b)performs atmost $O(n)$ Lookup operations and distance computations (25 distance computed for each new point)
- Let $X_i$ is an indicator $(0-1)$ random variable; ie.

$$X_i = \begin{cases} 1, & \text{if } i^{th} \text{ point causes } \delta \text{ to change} \\ 0, & \text{otherwise} \end{cases} \tag{4}$$

- Let $X$ is a random variable representing the **number of insert operations** performed in the Dictionary
- **Observations 3:** total number of insert operations is

  $X = n + \sum_{i=1}^{n} i.X_i$
  - the first $n$: Each point is inserted once when it is first encountered;
  - $\sum_{i=1}^{n} i.X_i$: reinsert all $i$ points when $\delta$ has changed due to $p_i$

# Time complexity (2)

**Observation 4:** For 0-1 random variable $X$,
$E[X] = 0.Pr[X = 0] + 1.Pr[X = 1] = Pr[X = 1]$
**Lemma:** $Pr[X_i = 1] \leq \frac{2}{i}$
**Proof:**

- Let minimum distance among $i$ points $p_1, p_2, \cdots p_i$ be determined by two points $p$ and $q$
- point $p_i$ can only cause the minumum distance to decrease if $p_i = p$ or $p_i = q$
- Now, the first $i$ points are in random order, thus, any of them is equally likely to be the last chosen point
- the probability that $p$ or $q$ is the last is $\frac{1}{i} + \frac{1}{i} = \frac{2}{i}$

# Time complexity (3)

**Compute** $E(X)$

$$
\begin{aligned}
&= E\left[n + \sum_{i=1}^{n} i.X_i\right] \\
&= n + E\left[\sum_{i=1}^{n} i.X_i\right] \\
&= n + \sum_{i=1}^{n} i.E\left[X_i\right] \\
&= n + \sum_{i=1}^{n} i.\frac{2}{i}, \text{ Lemma and Obs. 4} \\
&\leq 3n = O(n)
\end{aligned}
$$

In expectation, the randomized closest-pair algorithm requires $O(n)$ time plus $O(n)$ dictionary operations.

# References

 J. Kleinberg and E. Tardos, Algorithm Design,Pearson, Chapter 13.6,13.7

# The End